CS726: Assignment-3
Decoding Strategies for Large Language Models

Mrudul Jambhulkar - 21D070044
Saurav Raj - 21D070064

Contents

1 Introduction **3**

2 Task 0: Introduction to LLM Decoding Techniques **3**

2.1 Implementation Details 3

2.1.1 Greedy Decoding 4

2.1.2 Random Sampling with Temperature Scaling 4

2.1.3 Top-k Sampling 4

2.1.4 Nucleus Sampling 4

2.2 Experiments and Results 5

3 Task 1: Word-Constrained Decoding **5**

3.1 Implementation Details 5

3.1.1 Trie Data Structure 6

3.1.2 Decoding Algorithm 6

3.2 Experiments and Results 7

3.2.1 Analysis 7

4 Task 2: Staring into Medusa's Heads **7**

4.1 Approach 8

4.2 Results 8

5 References **8**

6 Individual Contribution **9**

Abstract

This report presents our work on CS726 Programming Assignment 3, focusing on the implementation and analysis of various decoding strategies for Large Language Models (LLMs). We explore techniques to enhance text generation performance, utilizing the Llama-2-7b-hf model and the IN22-Gen dataset for Hindi-to-English translation tasks. Our study evaluates the effectiveness of these strategies using standard metrics such as BLEU and ROUGE scores. Additionally, **we acknowledge the limited use of AI tools like ChatGPT for conceptual guidance and coding support, with all external sources properly cited in the report and code comments. All implementations adhere to the provided environment specifications.**

1 Introduction

Large Language Models (LLMs) have become a necessary tool while working with natural language processing, enabling a wide range of applications from text generation to machine translation. The decoding strategy used during inference significantly influences the quality, diversity, and efficiency of the generated outputs. In this assignment, we dive into advanced decoding techniques applied to the Llama-2-7b-hf model, evaluated on the IN22-Gen dataset for Hindi-to-English translation. Our work aims to understand how different decoding methods impact translation performance.

2 Task 0: Introduction to LLM Decoding Techniques

This section explores various decoding strategies for Large Language Models (LLMs) using the Llama-2-7b-hf model on the IN22-Gen dataset for Hindi-to-English translation. We implemented and analyzed four decoding techniques: **Greedy Decoding, Random Sampling with Temperature Scaling, Top-k Sampling, and Nucleus Sampling**. Each method was evaluated using BLEU and ROUGE scores to assess translation quality.

2.1 Implementation Details

We implemented the decoding strategies in the `TextGenerator` class, utilizing PyTorch and the Hugging Face Transformers library. The model processes input token IDs and generates output sequences, stopping when either the end-of-sequence (EOS) token (`self.eos_token_id`) is produced or the maximum output length (`self.max_output_len`) is reached.
Below is the detail for each method:

2.1.1 Greedy Decoding

Greedy Decoding selects the token with the highest probability at each step, defined as:

$$y_t = \arg\max_w P(w \mid y_{1:t-1}, x)$$

where $y_{1:t-1}$ are previously generated tokens, and x is the input prompt. The implementation iteratively appends the most probable token from the model's logits until the EOS token is generated or the maximum length is reached.

2.1.2 Random Sampling with Temperature Scaling

Random Sampling introduces stochasticity by sampling from a temperature-adjusted probability distribution:

$$P'(w \mid y_{1:t-1}, x) = \frac{P(w \mid y_{1:t-1}, x)^{1/\tau}}{\sum_{w' \in V} P(w' \mid y_{1:t-1}, x)^{1/\tau}}$$

where τ controls the distribution's sharpness. We experimented with $\tau \in \{0.5, 0.9\}$, sampling tokens using `torch.multinomial` until the stopping criteria are met.

2.1.3 Top-k Sampling

Top-k Sampling restricts sampling to the k most probable tokens:

$$V_k = \{w_1, w_2, \ldots, w_k\}, \quad \text{where } P(w_i) \geq P(w_{i+1}) \text{ for } i < k$$

$$P'(w) = \begin{cases} \frac{P(w)}{\sum_{w' \in V_k} P(w')} & \text{if } w \in V_k \\ 0 & \text{otherwise} \end{cases}$$

We tested $k \in \{5, 10\}$, normalizing probabilities over the top-k tokens and sampling from this subset.

2.1.4 Nucleus Sampling

Nucleus Sampling dynamically selects a token set V_p where the cumulative probability exceeds a threshold p:

$$V_p = \{w_1, w_2, \ldots, w_m\}, \quad \text{such that } \sum_{i=1}^{m} P(w_i) \geq p$$

$$P'(w) = \begin{cases} \frac{P(w)}{\sum_{w' \in V_p} P(w')} & \text{if } w \in V_p \\ 0 & \text{otherwise} \end{cases}$$

We evaluated $p \in \{0.5, 0.9\}$, sampling from the normalized nucleus distribution.

2.2 Experiments and Results

We evaluated each decoding strategy on the IN22-Gen dataset, reporting BLEU, ROUGE-1, ROUGE-2, and ROUGE-LCS scores. Results are summarized below:

Decoding Strategy	**BLEU**	**ROUGE-1**	**ROUGE-2**	**ROUGE-LCS**
Greedy Decoding	0.3097	0.3538	0.1297	0.2704
Random Sampling ($\tau = 0.5$)	0.2856	0.2929	0.1113	0.2387
Random Sampling ($\tau = 0.9$)	0.1996	0.1791	0.0550	0.1477
Top-k Sampling ($k = 5$)	0.2366	0.2267	0.0607	0.1738
Top-k Sampling ($k = 10$)	0.2200	0.2204	0.0538	0.1683
Nucleus Sampling ($p = 0.5$)	0.2706	0.2554	0.0905	0.1973
Nucleus Sampling ($p = 0.9$)	0.1957	0.1883	0.0469	0.1329

Table 1: Performance metrics for decoding strategies in Task 0.

Observations:
- **Greedy Decoding** achieved the highest BLEU (0.3097) and ROUGE-1 (0.3538) scores.
- **Random Sampling**: Lower $\tau = 0.5$ outperformed $\tau = 0.9$, with BLEU dropping from 0.2856 to 0.1996 as temperature increased, indicating increased randomness and reduced coherence.
- **Top-k Sampling**: $k = 5$ yielded better results (BLEU: 0.2366) than $k = 10$ (BLEU: 0.2200), suggesting a smaller k balances quality effectively.
- **Nucleus Sampling**: $p = 0.5$ outperformed $p = 0.9$ (e.g., BLEU: 0.2706 vs. 0.1957).

3 Task 1: Word-Constrained Decoding

This section implements a Word-Constrained Decoding technique, a variant of Grammar-Constrained Decoding, where the generation process is guided by a pre-provided list of words that must appear in the output. Using the Llama-2 model and the IN22-Gen dataset for Hindi-to-English translation, we designed a greedy decoding strategy that leverages this word list to enhance translation performance. The approach is evaluated against the baseline strategies from Task 0 using BLEU and ROUGE scores.

3.1 Implementation Details

The implementation is encapsulated in the `ConstrainedTextGenerator` class, which integrates a Trie data structure to enforce word constraints efficiently. Below, we outline the key components and the decoding process:

3.1.1 Trie Data Structure

To manage the word list, we constructed a Trie (prefix tree) with the following properties: - **Node Structure**: Each `TrieNode` contains a dictionary of children (mapping characters to nodes) and a boolean flag `is_end` indicating word completion. - **Insertion**: Words from the list are inserted with a leading space (e.g., " word") to ensure proper word boundary handling, accommodating tokenization variations in the LLM. The insertion process is:

For each character c in word w : if $c \notin$ node.children, add new node; move to node.children[c]

The final node is marked as `is_end = True`. - **Prefix Search**: The `search_prefix` method checks if a given prefix exists in the Trie, returning the corresponding node or `None` if absent. This enables validation of partial words during decoding.

Since the model may tokenize a single word into multiple tokens , the trie data structure can check whether the current prefix generated can form a valid continuation so that words in the word list can be formed and continues to complete the word .

3.1.2 Decoding Algorithm

The decoding process is a constrained greedy approach, selecting the most probable token that aligns with the word list at each step:

$$y_t = \arg \max_{w \in V_{\text{valid}}} P(w \mid y_{1:t-1}, x)$$

where V_{valid} is the subset of vocabulary tokens that form valid prefixes of words in the Trie, given the current prefix. The algorithm proceeds as follows:
1. **Initialization**: Start with the input token IDs and an empty generated sequence. Initialize the current prefix as a space (" ") to enforce word boundaries.
2. **Logits Generation**: Pass the current sequence to the model to obtain logits for the next token (`logits[:, -1, :]`).
3. **Token Validation**: For each token in the vocabulary: - Decode the token using the tokenizer (`tokenizer.decode([token_id])`), preserving spaces and special characters. - Append it to the current prefix and check if the resulting string is a valid prefix in the Trie using `search_prefix`. - Mark the token as valid in a boolean mask if a matching prefix exists.
4. **Constrained Selection**: Mask invalid tokens by setting their logits to $-\infty$, then select the token with the highest probability among valid options using `torch.argmax`.
5. **State Update**: - Append the selected token to the generated sequence. - Update the current prefix by concatenating the decoded token text. - If the prefix completes a word (i.e., the Trie node has `is_end = True`), reset the prefix to " " to start a new word.
6. **Stopping Criteria**: Halt if the EOS token is generated, the maximum length (10 tokens) is reached, or no valid tokens remain (all logits are $-\infty$).

This approach ensures that generated tokens incrementally build words from the provided list, adapting to the LLM's tokenization (e.g., single-token words vs. multi-token sequences).

3.2 Experiments and Results

We evaluated Word-Constrained Decoding on the IN22-Gen dataset, using a maximum output length of 10 tokens. The performance metrics are: - BLEU: 0.2880 - ROUGE-1: 0.3219 - ROUGE-2: 0.1663 - ROUGE-LCS: 0.2817

Decoding Strategy	**BLEU**	**ROUGE-1**	**ROUGE-2**	**ROUGE-LCS**
Greedy Decoding	0.3097	0.3538	0.1297	0.2704
Random Sampling ($\tau = 0.5$)	0.2856	0.2929	0.1113	0.2387
Top-k Sampling ($k = 5$)	0.2366	0.2267	0.0607	0.1738
Nucleus Sampling ($p = 0.5$)	0.2706	0.2554	0.0905	0.1973
Word-Constrained Decoding	0.2880	0.3219	0.1663	0.2817

Table 2: Performance metrics for Word-Constrained Decoding compared to Task 0 strategies.

3.2.1 Analysis

- **Comparison with Greedy Decoding**: Word-Constrained Decoding slightly underperforms Greedy Decoding in BLEU (0.2880 vs. 0.3097) and ROUGE-1 (0.3219 vs. 0.3538). However, it surpasses Greedy Decoding in ROUGE-2 (0.1663 vs. 0.1297) and ROUGE-LCS (0.2817 vs. 0.2704), indicating better bigram overlap and longer subsequence matching. This suggests that the constrained vocabulary aligns more closely with reference text structures at a phrase level.
- **Comparison with Stochastic Methods**: Compared to Random Sampling ($\tau = 0.5$), Top-k Sampling ($k = 5$), and Nucleus Sampling ($p = 0.5$), Word-Constrained Decoding achieves higher scores across most metrics (e.g., BLEU: 0.2880 vs. 0.2856, 0.2366, 0.2706). The deterministic nature, combined with word constraints, likely enhances coherence and relevance over purely stochastic approaches.
- **Impact of Word List**: The provided word list acts as an oracle, guiding the model toward relevant vocabulary. However, if the list excludes key translation terms or mismatches the tokenizer's subword units, it may constrain the model excessively, explaining the BLEU drop relative to Greedy Decoding.

4 Task 2: Staring into Medusa's Heads

Speculative decoding can predict several subsequent tokens in parallel, enabling faster inference compared to the traditional auto-regressive decoding. Medusa heads act as draft models to generate several tokens in parallel .

4.1 Approach

Single Head Decoding: This traditional approach uses a single LM head for autoregressive generation, producing one output sequence at a time. It ensures high-quality generation but can be computationally expensive.

Multi Head Decoding (Medusa Heads): Medusa leverages multiple heads in parallel to generate multiple candidate sequences, enhancing efficiency. Using beam search (Algorithm 1), the model evaluates and prunes candidates based on likelihood, retaining the most probable sequences. This approach balances speed and quality. The definition of the k-th head is :

$$p_t^{(k)} = \text{softmax}\left(W_2^{(k)} \cdot \text{SiLU}(W_1^{(k)} \cdot h_t) + h_t\right)$$

MEDUSA heads are extra decoding heads appended to the last hidden states of the original language model. Specifically, given the original model's last hidden states h_t at position t, we add K decoding heads to h_t and prediction of the k-th head is denoted as $p_t^{(k)}$. Key variables include the number of heads (S) and beam width (W), which impact speed and performance.

4.2 Results

We evaluated Single Head Decoding on the IN22-Gen dataset, measuring translation quality with BLEU and ROUGE scores and efficiency with Real Time Factor (RTF), defined as the ratio of inference time to output length. The results are:

- BLEU: 0.2921 - ROUGE-1: 0.3963 - ROUGE-2: 0.1483 - ROUGE-LCS: 0.3177 - RTF: 0.0566

Decoding Strategy	BLEU	ROUGE-1	ROUGE-2	ROUGE-LCS	RTF
Single Head Decoding	0.2921	0.3963	0.1483	0.3177	0.0566

Table 3: Performance metrics for Single Head Decoding.

5 References

- ChatGPT
- `https://huggingface.co/blog/mlabonne/decoding-strategies`
- `https://www.geeksforgeeks.org/trie-insert-and-search/`
- `https://pytorch.org/docs/stable/index.html`
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024.

6 Individual Contribution

Both team members actively discussed and collaboratively wrote the code files, along with helping each other with part of the code task. Specific tasks handled individually were:

- **task0.py:**
  - → Mostly handled by the member – **Saurav**
- **task1.py**
  - → Mostly handled by the member – **Mrudul**
- **task2.py**
  - → Both collectively tried to implement the single and multi-head decoding strategies but couldn't generate the correct logic for multi-head decoding part.– **Mrudul**, **Saurav**